LAB IMPLEMENTATION 6

Creation and Deployment of a Ballot Smart Contract

BC

Aim

To create, compile, deploy, and interact with a Ballot smart contract using Solidity and Remix IDE. To perform voting and delegation transactions and observe the contract state through Remix.

Theory

A Solidity smart contract is a program deployed on a blockchain whose state and functions can be accessed through transactions and calls. The Ballot contract models voting with a Voter structure, a Proposal structure, an address-based mapping, and a dynamic array of proposals. The constructor initializes the chairperson, gives the chairperson voting weight, and stores the supplied proposal names.

The contract uses require statements to enforce conditions before state changes. storage references persistent contract state, while memory is used for temporary data such as the constructor input. The experiment changes proposal names from bytes32 to string so readable names can be supplied directly during deployment. Functions such as vote, delegate, winningProposal, and winnerName provide the main voting and result operations.

Steps

1. Open Remix IDE and access the contracts workspace.
2. Open Ballot.sol and inspect the Ballot contract, its structures, mapping, proposal array, and functions.
3. Change the proposal name type from bytes32 to string and use the modified Ballot.sol program.
4. Compile Ballot.sol in the Solidity Compiler panel and verify that the Ballot contract is compiled.
5. Open Deploy & Run Transactions, select Remix VM, and provide the proposal names Proposal A, Proposal B, and Proposal C.
6. Deploy the contract and observe the deployment transaction in the transaction list.
7. Use the available contract functions to inspect the author, proposals, winner, and voting-related state.
8. Perform the required voting or delegation interaction and observe the resulting contract calls and transactions.

Code

```
// SPDX-License-Identifier: GPL-3.0

/*
 * ============================================================
 * AIM     : Creation and Deployment of Smart Contract and
 *           Transactions using Solidity and Remix IDE
 *  CO     : CO3, CO5, CO6
 *
 * Author : Afnan Kazi
 * Roll No: 14
 *
 * Modification (by Afnan Kazi, Roll No 14):
 * The original Remix Ballot.sol stores proposal names as
 * bytes32. In this version the proposal name has been
 * changed from bytes32 -> string, so that readable names
 * like "Proposal A" can be passed directly at deployment.
 * ============================================================
 */

pragma solidity >=0.7.0 <0.9.0;

/// @title Voting with delegation. (Modified by Afnan Kazi | Roll No: 14)
contract Ballot {

    // Author details stored on-chain as proof of ownership
    string public constant AUTHOR = "Afnan Kazi";
    string public constant ROLL_NO = "14";

    // This declares a new complex type which will be used for variables later.
    // It will represent a single voter.
    struct Voter {
        uint weight;      // weight is accumulated by delegation
        bool voted;       // if true, that person already voted
        address delegate; // person delegated to
        uint vote;        // index of the voted proposal
    }

    // This is a type for a single proposal.
    struct Proposal {
        string name;      // CHANGED: bytes32 -> string (Afnan Kazi, Roll 14)
        uint voteCount;   // number of accumulated votes
    }

    address public chairperson;

    // This declares a state variable that stores a 'Voter' struct for each possible address.
    mapping(address => Voter) public voters;

    // A dynamically-sized array of 'Proposal' structs.
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot to choose one of 'proposalNames'.
     * @param proposalNames names of proposals (now as string[] instead of bytes32[])
     *
     * Example input while deploying in Remix:
     * ["Proposal A","Proposal B","Proposal C"]
     */
    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;

        // For each of the provided proposal names,
        // create a new proposal object and add it to the end of the array.
        for (uint i = 0; i < proposalNames.length; i++) {
            // 'Proposal({...})' creates a temporary Proposal object and
            // 'proposals.push(...)' appends it to the end of the array.
            proposals.push(Proposal({
                name: proposalNames[i],
                voteCount: 0
            }));
        }
    }

    /**
```

```
76       * @dev Give 'voter' the right to vote on this ballot. May only be called by 'chairperson'.
77       * @param voter address of voter
78       */
79      function giveRightToVote(address voter) public {
80          require(
81              msg.sender == chairperson,
82              "Only chairperson can give right to vote."
83          );
84          require(
85              !voters[voter].voted,
86              "The voter already voted."
87          );
88          require(voters[voter].weight == 0);
89          voters[voter].weight = 1;
90      }
91
92      /**
93       * @dev Delegate your vote to the voter 'to'.
94       * @param to address to which vote is delegated
95       */
96      function delegate(address to) public {
97          Voter storage sender = voters[msg.sender];
98          require(sender.weight != 0, "You have no right to vote");
99          require(!sender.voted, "You already voted.");
100         require(to != msg.sender, "Self-delegation is disallowed.");
101
102         // Forward the delegation as long as 'to' also delegated.
103         while (voters[to].delegate != address(0)) {
104             to = voters[to].delegate;
105
106             // We found a loop in the delegation, not allowed.
107             require(to != msg.sender, "Found loop in delegation.");
108         }
109
110         Voter storage delegate_ = voters[to];
111
112         // Voters cannot delegate to accounts that cannot vote.
113         require(delegate_.weight >= 1);
114
115         // Since 'sender' is a reference, this modifies 'voters[msg.sender]'.
116         sender.voted = true;
117         sender.delegate = to;
118
119         if (delegate_.voted) {
120             // If the delegate already voted,
121             // directly add to the number of votes for their proposal.
122             proposals[delegate_.vote].voteCount += sender.weight;
123         } else {
124             // If the delegate did not vote yet, add to her weight.
125             delegate_.weight += sender.weight;
126         }
127     }
128
129     /**
130      * @dev Give your vote (including votes delegated to you) to proposal 'proposals[proposal].name'.
131      * @param proposal index of the proposals array
132      */
133     function vote(uint proposal) public {
134         Voter storage sender = voters[msg.sender];
135         require(sender.weight != 0, "Has no right to vote");
136         require(!sender.voted, "Already voted.");
137         sender.voted = true;
138         sender.vote = proposal;
139
140         // If 'proposal' is out of the range of the array,
141         // this will throw automatically and revert all changes.
142         proposals[proposal].voteCount += sender.weight;
143     }
144
145     /**
146      * @dev Computes the winning proposal taking all previous votes into account.
147      * @return winningProposal_ index of winning proposal in the proposals array
148      */
149     function winningProposal() public view
150             returns (uint winningProposal_)
151     {
152         uint winningVoteCount = 0;
153         for (uint p = 0; p < proposals.length; p++) {
```

```
154             if (proposals[p].voteCount > winningVoteCount) {
155                 winningVoteCount = proposals[p].voteCount;
156                 winningProposal_ = p;
157             }
158         }
159     }
160
161     /**
162      * @dev Calls winningProposal() function to get the index of the winner
163      *         contained in the proposals array
164      *         and then returns the name of the winner.
165      * @return winnerName_ the name of the winner (CHANGED: bytes32 -> string)
166      */
167     function winnerName() public view
168             returns (string memory winnerName_)
169     {
170         winnerName_ = proposals[winningProposal()].name;
171     }
172 }
```

Listing 1. Ballot.sol

Output

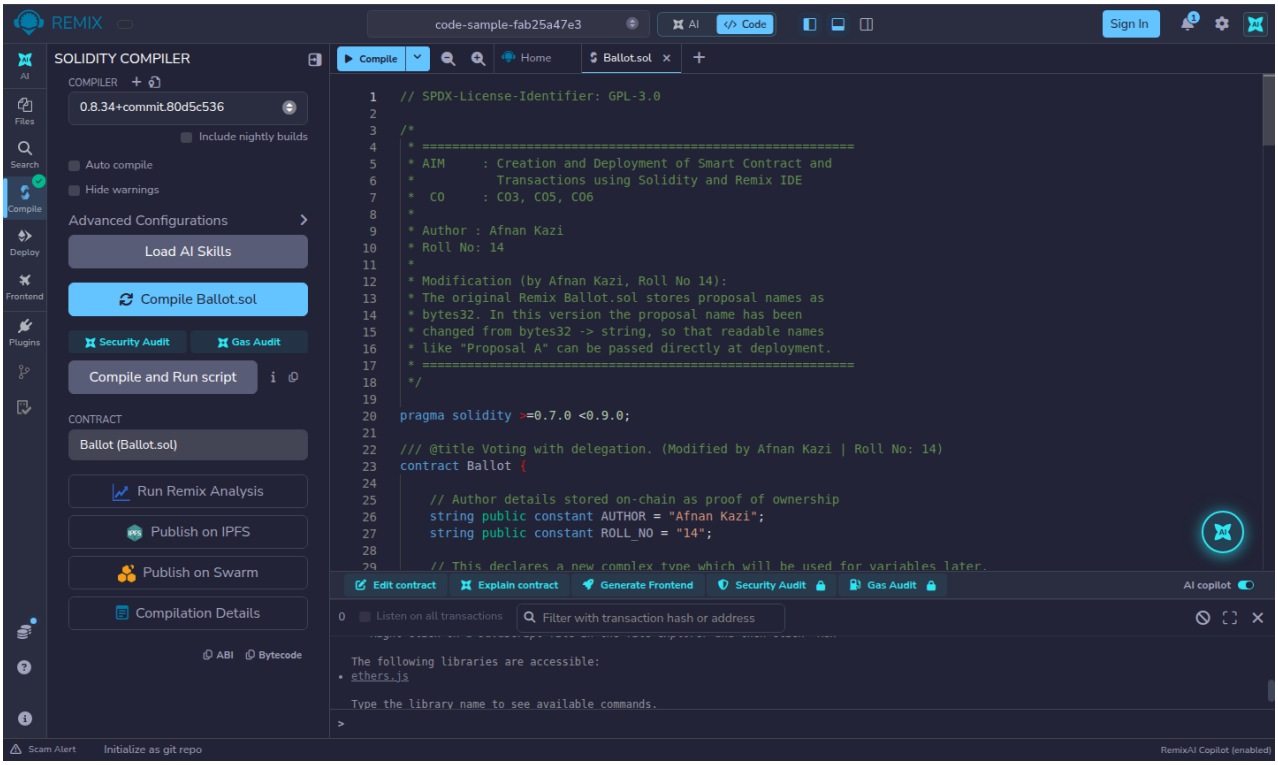


Fig. 1. Compiled Ballot contract details

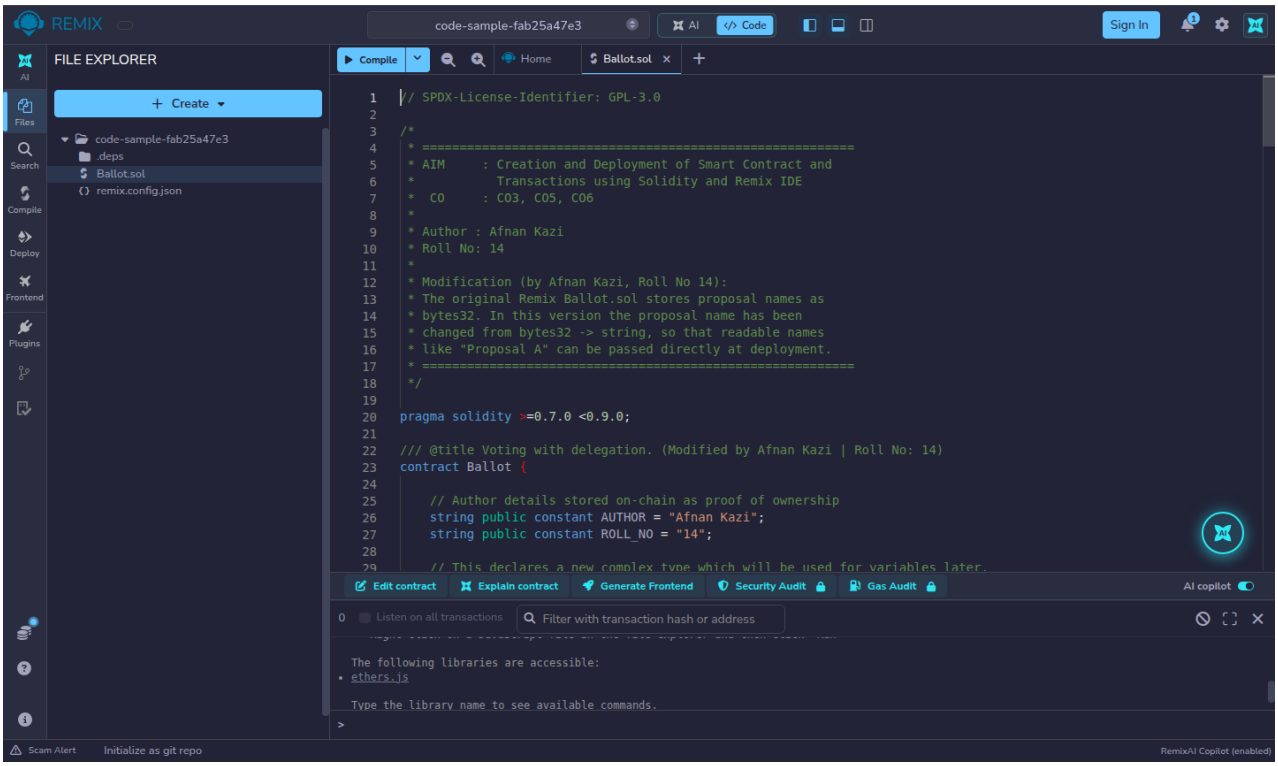


Fig. 2. Ballot source code in Remix IDE

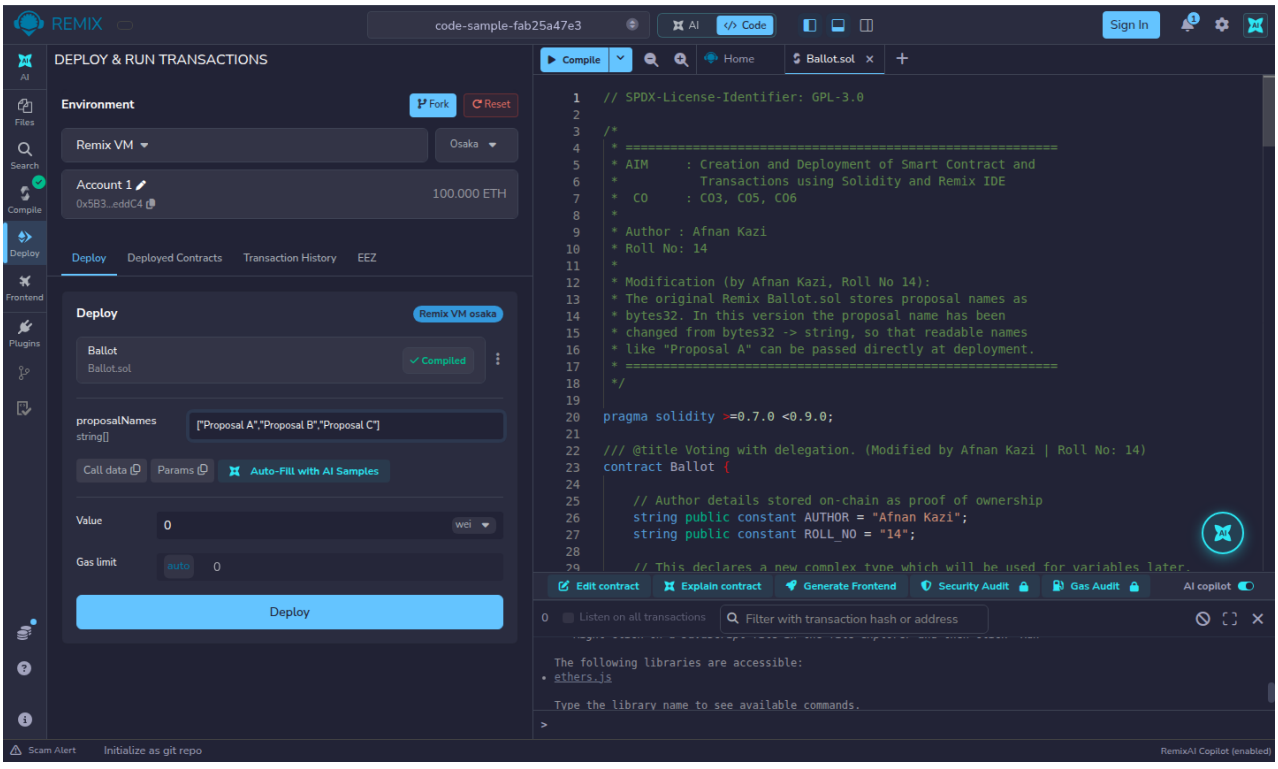


Fig. 3. Deployment form with proposal names

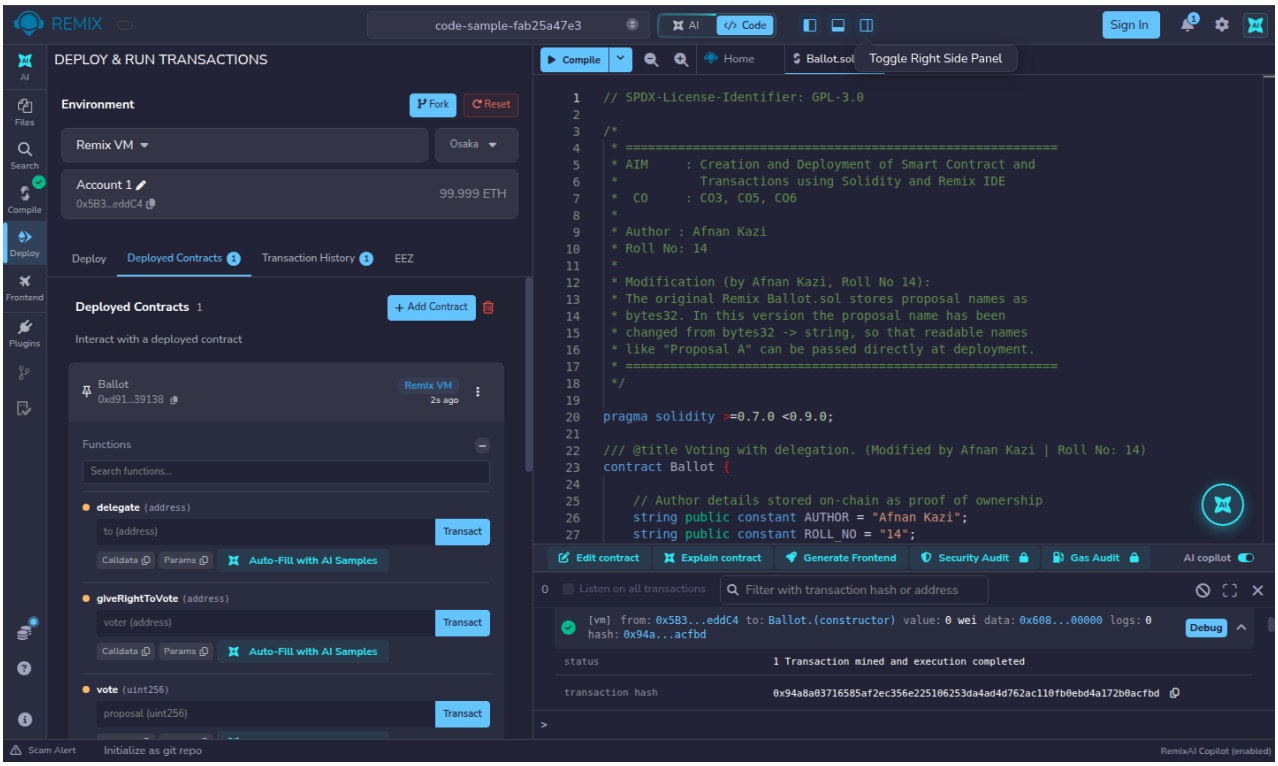


Fig. 4. Successful Ballot deployment transaction

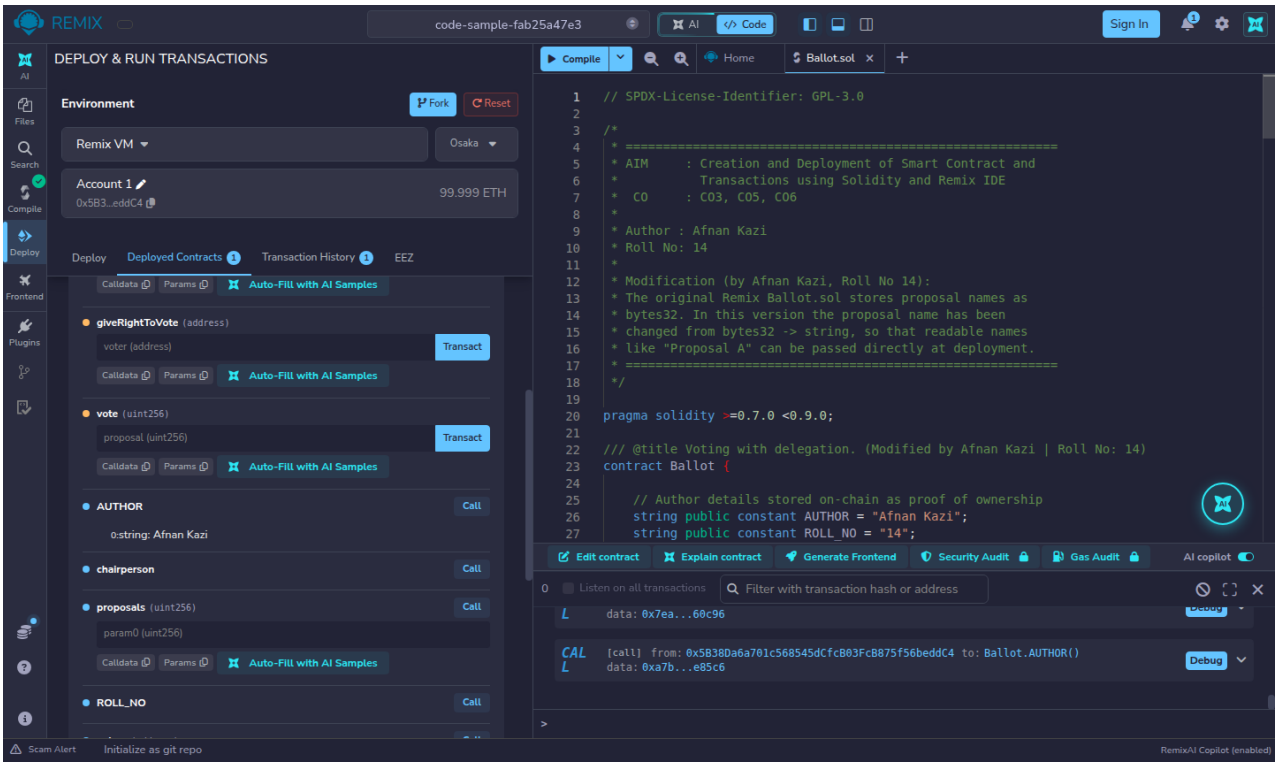


Fig. 5. Author value returned by contract call

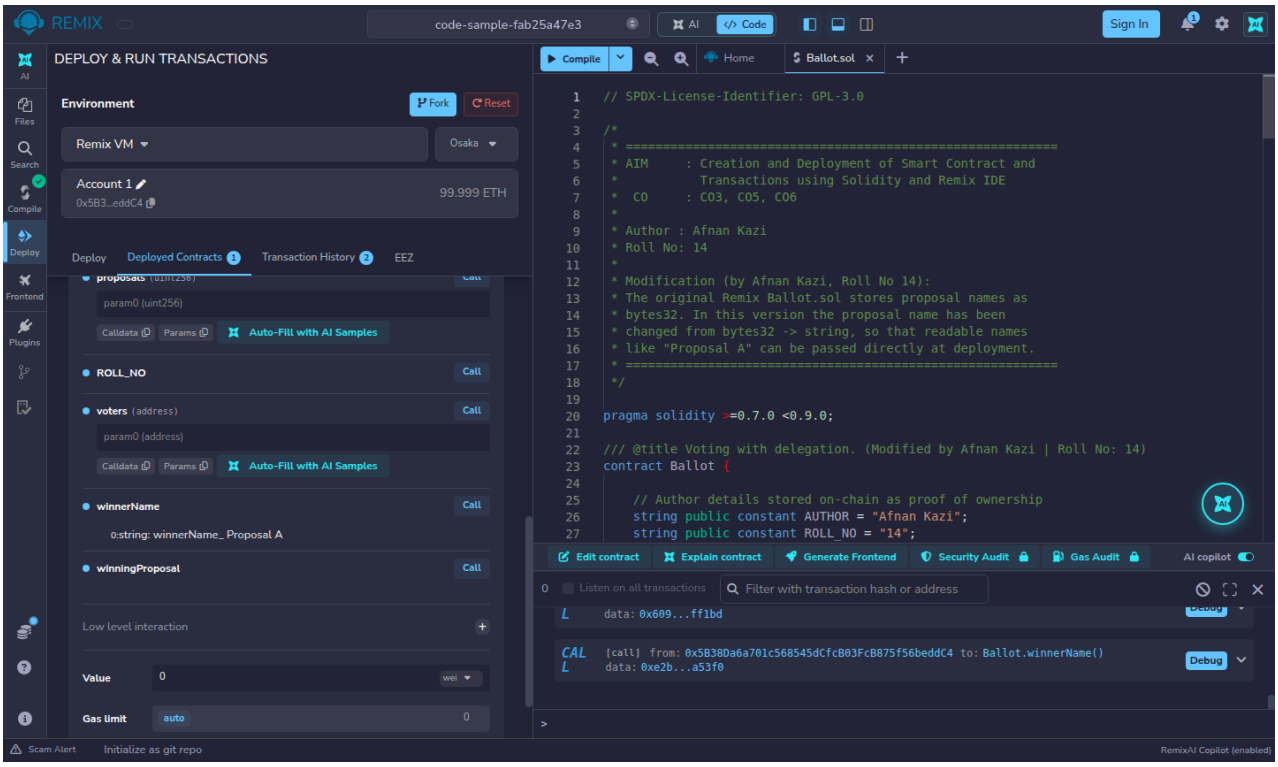


Fig. 6. Winner name returned by contract call

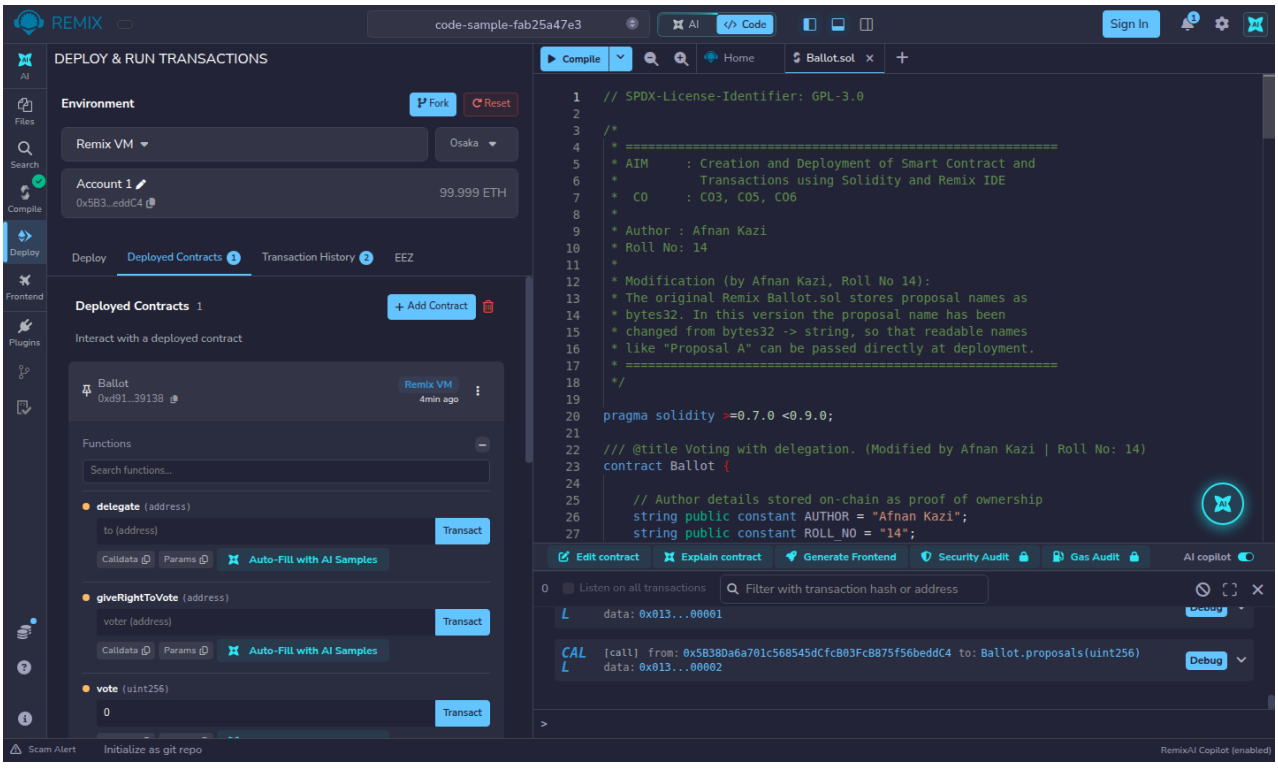


Fig. 7. Delegation and voting functions available

Conclusion

The Ballot smart contract was compiled and deployed in Remix IDE using the modified proposal-name representation. The deployment used readable proposal names and produced a successful contract transaction in the Remix VM. Contract calls displayed the stored author information and the winner name, while the deployed interface exposed voting and delegation functions. The experiment demonstrated the deployment and transaction workflow of a Solidity voting contract in Remix.